软件工程

实验三：软件实现与构建

活动票务与报名系统（票集）

陆知渔 241880173

2026 年 11 月 2 日

一、实验概述

1.1 实验任务与完成情况

本次实验的任务是把实验二完成的 UML 设计实现为可运行、可测试的软件，并完成代码风格检查、大模型辅助开发与 Git 远程托管。实现沿用实验一、实验二确定的“活动票务与报名系统”，以参与者、组织者、检票人员、平台管理员四类角色为主线，覆盖活动审核发布、报名与候补、检票、统计导出、通知与审计日志。各任务的要求与完成情况如表 1-1。

任务	实验要求	完成情况
UML 到代码	实现超过 500 行代码，遵守语言风格	产品代码 1724 行，测试 243 行，合计 1967 行；Pylint 评分 9.92/10
编译与运行	使用合适的编译器或 IDE 编译并运行	Python 3.10 解释执行，提供端到端演示脚本与 12 个单元测试
大模型辅助	使用大模型辅助开发并分析优缺点	共 5 次主要使用，覆盖初稿、持久化、报名核心、统计导出、测试与 lint
Git 管理	建立本地仓库并托管至远程仓库	GitHub 远程仓库，lab3 分支跟踪 origin/lab3，5 个阶段标签

表 1-1 实验任务与完成情况

1.2 运行环境与技术选型

实现采用 Python 3.10 + SQLite，界面为命令行。具体版本见表 1-2。

项目	版本或说明
操作系统	Ubuntu 22.04（Linux 6.8.0）
语言与解释器	Python 3.10.12
数据库	SQLite 3.37.2，经标准库 sqlite3 访问
代码风格工具	Pylint 4.1.3（Astroid 4.3.4）
第三方依赖	无；仅使用 Python 标准库
界面形式	命令行菜单，按角色进入不同菜单

表 1-2 运行环境

这样选型有四点考虑。第一，实验不要求图形界面，命令行可以把精力集中在状态机、权限、并发与数据一致性上，也便于用脚本复现演示。第二，SQLite 支持事务、外键、部分唯一索引与检查约束，足以承载实验二设计的约束，并且单文件、无需部署，测试时可为每个用例新建临时库。第三，Python 标准库已经包含 sqlite3、hashlib、csv、unittest 等模块，不引入第三方依赖，评审者可以直接运行。第四，Python 没有独立编译步骤，因此把“构建”落在字节码编译、单元测试与静态检查三道关卡上，运行结果和检查结果都可复现。

二、从 UML 设计到代码

2.1 分层结构与模块映射

实验二的组件图把系统分为表现层、应用服务层、领域层与基础设施层，并规定依赖只能自上而下。实现保持这一分层：main.py 只负责菜单与输入输出，业务用例放在 events.py、registration.py、checkin.py、reports.py、notifications.py，共享状态、角色与异常放在 models.py，数据库、事务、密码哈希与审计落在 db.py 和 auth.py。模块之间没有循环导入，服务函数统一接收数据库连接 conn，由调用方决定事务边界与测试数据库。映射关系见表 2-1。

实验二设计（组件图）	实现文件	职责
表现层（四类角色页面）	main.py	登录循环、四套角色菜单、输入解析、异常提示
活动与审核服务	events.py	活动、场次、票种、审核、取消、缺席标记
报名与候补服务	registration.py	报名、幂等、候补、递补、取消、凭证码
检票服务	checkin.py	场次授权校验、凭证核验、重复签到识别
统计导出、审计查询	reports.py	名单筛选、容量与签到率、CSV、平台汇总、日志
通知服务	notifications.py	通知记录、失败状态、人工补发
领域模型与业务策略	models.py	角色、六种活动状态、七种报名状态、异常、截止判定
数据库、事务、审计	db.py	表结构、连接参数、演示数据、操作日志
认证与授权	auth.py	登录校验、角色检查

表 2-1 分层结构与模块映射

2.2 类图到数据模型

实验二的领域类图（图 2-1）给出实体、状态枚举与关联。实现把每个实体落为一张表，把状态枚举落为 models.py 中的字符串常量，把关联落为外键，把“不能重复”的规则落为唯一索引与部分唯一索引。映射关系见表 2-2。

类图中的元素	实现位置	落库方式
User、Role	users 表，models.ROLE_*	account 唯一；role 存角色名；密码存 PBKDF2 摘要与盐
Event、EventState	events 表，models.EVENT_*	外键 organizer_id；state 存六种活动状态
EventSession	sessions 表	外键 event_id；capacity 有 CHECK 约束
TicketType	ticket_types 表	外键 session_id；capacity 有 CHECK 约束
Registration、RegistrationState	registrations 表	request_id 唯一；有效报名部分唯一索引
WaitlistEntry、WaitlistState	waitlist 表	同一用户在同一场次只能有一条“排队中”记录

类图中的元素	实现位置	落库方式
Credential	credentials 表	registration_id 唯一，code 唯一
CheckInRecord	checkins 表	成功与重复签到都留痕
OperationLog	operation_logs 表	操作人、动作、对象、结果、时间
Notification	notifications 表	状态、重试次数、关联报名
场次授权关系（时序图）	session_checkers 表	场次与检票人员多对多

表 2-2 类图元素与数据库表的对应

实现时对设计图做了三处调整，均出于“让约束可执行”的目的。第一，实验二类图的 EventSession 与 TicketType 都带 occupied 字段，实现没有保存这一冗余计数，而是在使用时按“已报名、已签到”两种状态实时 COUNT。原因是占用数会在报名、取消、递补、取消活动、标记缺席等多条路径变化，冗余字段一旦漏改就会与记录不一致；实时计数虽然多一次查询，但天然与事实一致。第二，实验二的 Event 带 version 乐观锁，实现改用 SQLite 的 BEGIN IMMEDIATE 写锁并设置 busy_timeout = 5000，让报名写入串行化，省去每次写入的版本比较；并发测试验证最后一张票只会被一个请求拿到。第三，类图的 Registration 有七种状态，实现把“候补中、候补失效”放到 waitlist 表，因为候补期间还没有报名记录；registrations 保存“已报名、已取消、已签到、缺席、活动取消”，两表合起来覆盖七种业务状态。

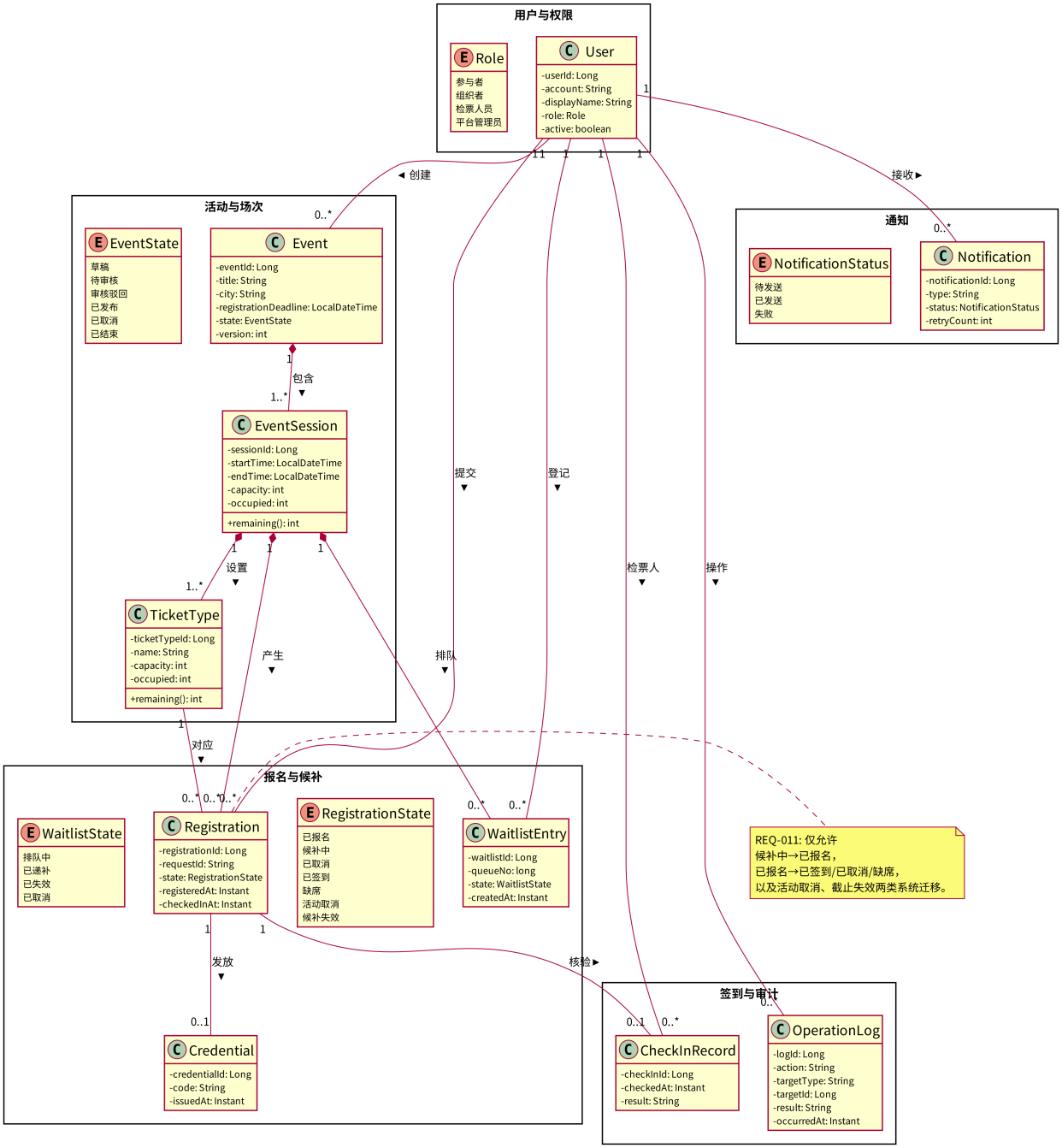


图 2-1 实验二领域类图（实现依据）

2.3 服务类图与时序图到函数

实验二的服务类图把报名拆成 RegistrationService、IdempotencyService、DeadlinePolicy、TransactionManager、WaitlistManager、ServerClock 等协作对象。实现没有为每个对象建类，而是用模块函数与显式事务表达同一套协作，见表 2-3。这样做的理由是：用例边界在 Python 中由模块与函数已经表达得足够清楚；数据库连接作为参数传入，使每个测试可以换用独立临时库；显式的事务块（BEGIN IMMEDIATE / COMMIT / ROLLBACK）比框架代理更容易逐行审查。代价是部分函数参数较多，Pylint 给出了提示，处理方式见第 5.4 节。

实验二的服务类	实现	说明
RegistrationService.register	registration.register	报名主用例，含权限、幂等、截止、容量与事务
RegistrationService.cancel	registration.cancel_registration	取消并在同一事务内尝试递补
IdempotencyService	registrations.request_id 唯一索引 + 函数开头查重	重复请求返回首次结果
DeadlinePolicy、ServerClock	models.deadline_open、now_text	统一使用服务端时间
TransactionManager	db.connect + BEGIN IMMEDIATE / COMMIT / ROLLBACK	写操作显式开事务
WaitlistManager	join_waitlist、_promote	排队、取队首、截止后失效
AuthorizationService	auth.require_role + 各服务的归属检查	角色与数据范围双重校验

表 2-3 服务类与函数的对应

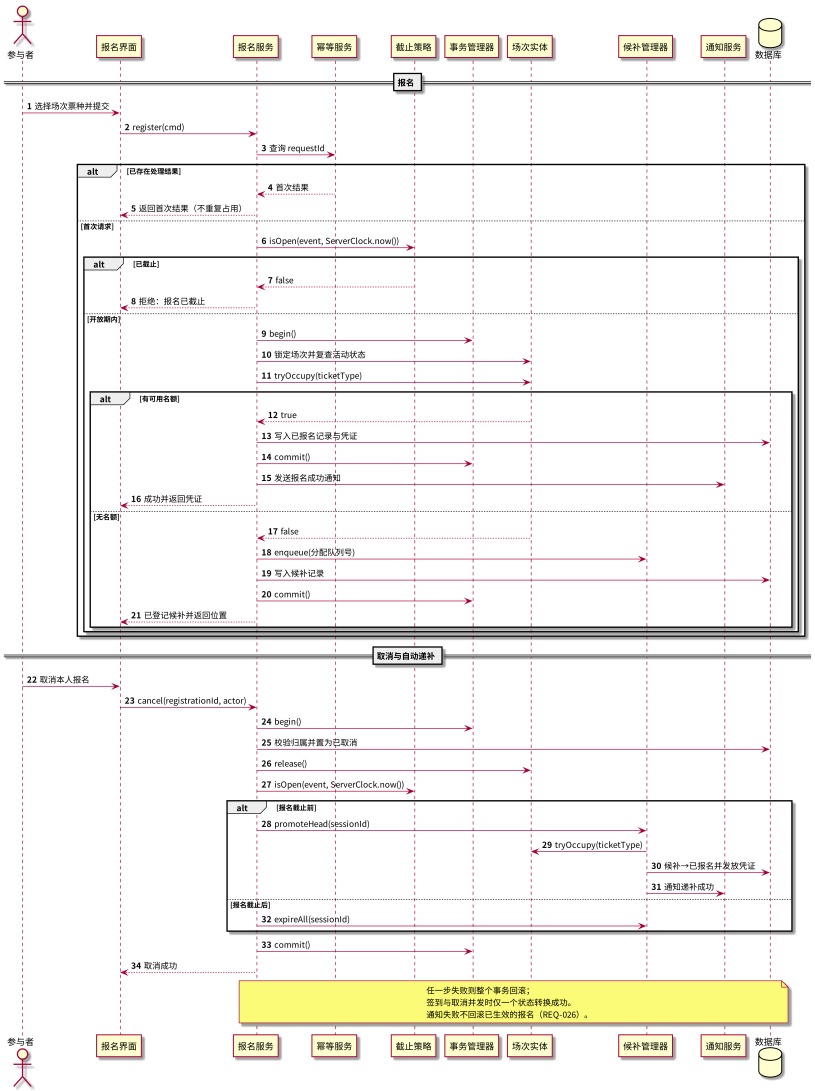


图 2-2 实验二报名与递补时序图（实现依据）

实验二的报名时序图（图 2-2）分为“报名”和“取消与自动递补”两段。实现的调用顺序与图一致：报名先查 requestId，再判断活动状态、截止时间与场次是否结束，然后进入事务，在事务内重新

读取活动状态、检查重复报名、比较场次与票种两级容量，最后插入报名与凭证、写日志、提交，通知在提交之后单独写入。取消报名的路径同样先开事务，校验归属与状态，置为已取消，若尚未截止则取候补队首递补，提交后再发送取消与递补通知。

2.4 状态图到状态约束

实验二的状态图只允许七种转换，实现用三层机制保证：(1) 服务层先读取当前状态，非法转换立即抛出 ServiceError；(2) 关键写在事务内再次读取状态，防止检查与写入之间被其他事务改变；(3) 数据库部分唯一索引兜底，即使应用层漏检也不会产生两条有效报名或两条排队候补。以报名为例，只有“活动已发布、未截止、场次未结束、本场次无有效报名、场次与票种均有余量”才插入“已报名”；取消只允许“已报名”变“已取消”；检票只允许“已报名”变“已签到”；取消活动把“已报名”改为“活动取消”，已签到保持不动；场次结束后把“已报名”改为“缺席”。这些判断分别位于 registration.register、registration.cancel_registration、checkin.check_in 与 events.cancel_event、events.finish_due_sessions。

2.5 代码规模

实现按模块拆分，规模统计见表 2-4。统计使用 wc -l，包含注释与空行；若只计非空行，产品代码 1528 行、含测试 1734 行。无论按哪种口径，均明显超过实验要求的 500 行。

文件	行数	职责
code/main.py	466	命令行入口与四类角色菜单
code/events.py	367	活动、场次、票种与生命周期
code/registration.py	294	报名、候补、递补、取消
code/db.py	229	表结构、连接、演示数据、审计
code/reports.py	164	名单、统计、CSV、汇总与日志查询
code/checkin.py	98	检票核验
code/models.py	48	角色、状态、异常、时间策略
code/notifications.py	35	通知记录与补发
code/auth.py	23	登录与角色校验
产品代码小计	1724	不含测试、README 与演示脚本
code/tests/test_services.py	243	核心规则与并发测试
合计	1967	非空行 1734

表 2-4 源代码规模

三、关键实现

3.1 身份认证与权限隔离

登录在 auth.login 中完成：按账号取出用户与盐，用 PBKDF2-HMAC-SHA256 迭代 100000 次重新计算摘要并比较，账号不存在与密码错误返回同一提示，避免泄露账号是否存在。演示数据中的初始密码由 db.add_user 加盐后写入。权限分两层：第一层是角色，auth.require_role 与各用例入口检查角

色，例如只有参与者能报名、只有管理员能审核、只有检票人员能核验；第二层是数据归属，events._check_owner 限制组织者只能管理自己创建的活动，reports._check_view 限制名单与统计只能由活动所有者或管理员查看，检票则通过 session_checkers 校验该场次是否分配给当前检票人员。参与者的报名查询始终带 user_id 条件，不能读取他人记录。

3.2 活动生命周期

活动有草稿、待审核、审核驳回、已发布、已取消、已结束六种状态。组织者创建活动时只能生成草稿，同时创建首个场次与默认票种；草稿或驳回状态可提交审核；管理员只能审核待审核活动，通过后变为已发布，驳回后组织者可以修改再交。只有草稿与驳回状态允许新增场次和票种，已发布活动要改结构必须先取消重建，避免在售活动中途改变容量口径。取消活动在一个事务内完成三件事：活动置为已取消、未签到的报名置为活动取消、排队中的候补置为失效；已签到记录与日志原样保留。启动时以及每次进入系统时运行 expire_waitlists 与 finish_due_sessions：已过截止时间的候补失效；已结束场次中仍为“已报名”的记录标为缺席；一个活动的所有场次都结束后，活动置为已结束。

3.3 报名、幂等与并发

报名是最复杂的用例，入口为 registration.register。它先检查角色、场次与票种的归属关系，再按 request_id 查重：同一请求号已存在时直接返回首次结果，若请求号属于其他用户则拒绝。随后检查活动已发布、报名未截止、场次未结束，然后开启 BEGIN IMMEDIATE 事务。事务内重新读取场次与活动状态，检查该用户在本场次是否已有有效报名，比较场次容量与票种容量两级余量，插入报名与凭证，写操作日志，提交。通知在提交后写入，不参与业务事务。核心骨架如下。

```
conn.execute("BEGIN IMMEDIATE")
try:
    fresh = get_session(conn, session_id)          # re-read inside txn
    if fresh["event_state"] != EVENT_PUBLISHED:
        raise ServiceError("活动状态已变化，报名失败")
    duplicate = ...                                # one active registration per session
    if (session_occupied(conn, session_id) >= fresh["capacity"]
            or ticket_occupied(conn, ticket_type_id) >= ticket["capacity"]):
        raise ServiceError("名额已满，请选择候补")
    registration_id, code = _insert_registration(...)
    log_action(conn, user["id"], "报名", "registration", registration_id, "成功")
    conn.execute("COMMIT")
except Exception:
    conn.execute("ROLLBACK")
    raise
```

代码 3-1 报名事务骨架（节选）

并发安全不只靠“先查后插”。数据库上还有两道约束：registrations.request_id 唯一，保证同一个请求最多生成一条报名；部分唯一索引 ux_active_registration 在 state IN ('已报名','已签到') 时对 (user_id, session_id) 唯一，保证一个人在同一场次最多一条有效报名。加上 BEGIN IMMEDIATE 在事务开始时取得写锁、busy_timeout 在锁冲突时等待而不是立即失败，两个并发请求抢最后一个名额时只有一个能提交。

3.4 候补与自动递补

候补只在“场次或票种确实已满”时允许，若还有余量会提示直接报名，避免出现“有票却排队”。join_waitlist 在事务内检查是否已有有效报名、是否已在队列中，再插入排队记录并返回位置。取消

报名时，若尚未到截止时间，_promote 取同场次、同票种的最早一条排队记录，复查余量与该用户是否已有有效报名，然后插入新报名与凭证、把候补置为已递补；若已过截止时间，则把该场次所有排队候补置为失效，不再递补。候补不占名额，只有已报名与已签到计入占用。

3.5 检票

checkin.check_in 在一个事务内查询凭证码，依次判断凭证是否存在、该场次是否分配给当前检票人员、活动是否已取消或结束、活动是否已发布、报名当前状态。重复核验时只返回“该凭证已签到”，并向 checkins 表写一条“重复签到”记录，签到计数不增加；状态不是“已报名”时拒绝签到；成功时把报名改为已签到、写入签到时间与签到记录。成功与失败都会写操作日志，便于管理员追溯。

3.6 统计、导出、通知与审计

统计按场次与票种输出容量、已报名、候补、剩余、签到、缺席和签到率。签到率严格按需求计算为“已签到 ÷（已签到 + 缺席）”，分母为零时显示“暂无数据”，而不是用全部报名数作分母。CSV 导出先写同目录的临时文件，成功后再用 os.replace 原子替换目标文件；任何写入失败都会删除临时文件、记录失败日志并抛出错误，不会留下一个不完整却看起来成功的文件。通知在业务事务提交后创建，记录“已发送/失败”和重试次数；管理员可以补发失败通知，补发只改通知状态并写日志，不回滚已经成功的报名。操作日志记录创建、修改、审核、取消、报名、递补、检票、导出、补发等关键操作，包含操作人、动作、对象、结果与时间，失败操作同样保留。

四、大模型辅助开发

4.1 使用原则

本次实验把大模型定位为“初稿生成器与代码审查员”，而不是设计决策者。每次使用都遵循三点：输入携带实验二的具体元素（类名、状态、时序步骤）和可验证的约束（不得超卖、重复请求返回首次结果、签到率分母等）；要求产物是可直接运行或可直接渲染的结果，而不是泛泛的建议；生成后必须通过运行、单元测试、Pylint 或人工逐行阅读中的至少一种验证。AI 初稿与每次人工修改分开提交，并用 Git 标签标出阶段，使修改过程可复盘。

4.2 使用记录

AI-01 内存原型初稿（标签 lab3-ai-draft）

使用场景：把实验二的类图、报名时序图和主要业务规则整理成提示，要求生成一个可直接运行的单文件 Python 原型，先跑通“登录—浏览活动—报名—查看凭证—检票—统计”的主流程。

生成结果：得到 216 行的单文件原型，使用字典与列表在内存中保存账号、活动与报名，包含四类账号登录、活动创建与浏览、报名、我的报名、检票等菜单。

优点：生成速度快，菜单结构与实验二的用例划分基本一致，能立即运行，适合作为讨论与拆分的起点。

缺点：数据只在内存，退出即丢失；创建活动直接就是“已发布”，绕过了审核；报名只比较人数，没有截止时间、重复报名、请求幂等；报名只有“已报名”一个状态；没有候补、没有凭证唯一性、没有角色数据范围；没有事务，无法承载并发；所有代码在一个文件，难以测试与定位。

修改优化后的结果：人工决定保留菜单骨架，废弃内存实现，按类图拆成 db、models、auth、events、main 五个模块并接入 SQLite，见 AI-02 与标签 lab3-rev-1。

AI-02 SQLite 持久化与活动生命周期（标签 lab3-rev-1）

使用场景：要求模型按类图建立数据库表，把原型改为持久化，并实现活动、场次、票种、审核与角色校验。

生成结果：给出 11 张表的建表语句、演示账号与两场演示活动；实现活动草稿、提交审核、通过、驳回、取消，以及登录与角色检查；代码量约 900 行，拆分为多个模块。

优点：表结构与类图对应度高；演示数据让程序首次运行就有内容；关键写入已经用事务包裹；模块拆分合理。

缺点：初稿把占用数做成表字段并在报名、取消时手工加减，容易漏改；密码使用简单哈希，未加盐与迭代；审核与编辑缺少活动归属检查；调整票种容量时可能降到已占用数以下；没有部分唯一索引。

修改优化后的结果：人工把占用改为按状态实时计数；密码改为 PBKDF2-HMAC-SHA256 加盐十万轮；增加活动归属检查与容量不低于占用的双重校验；补上 request_id 唯一索引与有效报名、排队候补两个部分唯一索引；连接增加外键开关与 busy_timeout。结果见提交 9c5f238。

AI-03 报名、候补、递补与检票（标签 lab3-rev-2）

使用场景：要求依据报名时序图实现报名、取消、候补、自动递补、检票和通知。

生成结果：生成 registration.py、checkin.py、notifications.py，覆盖报名、候补排队、取消后递补、凭证码生成、签到与重复签到提示。

优点：主流程完整；凭证码使用安全随机源；候补队列与递补逻辑清楚；检票把多种失败原因分别返回。

缺点：初稿在事务外判断名额，两个线程可能同时通过；截止时间用字符串直接比较，格式变化时不可靠；重复签到会重复计数；有名额时仍允许登记候补；通知写在业务事务内，通知失败会连带回滚报名。

修改优化后的结果：把所有名额判断与插入放进同一个 BEGIN IMMEDIATE 事务，并在事务内复查活动状态；时间统一经 now_text 与 parse_time 处理；重复签到只提示并写一条“重复签到”记录；有余量时拒绝候补并提示直接报名；通知移到提交之后。并发测试中两个线程抢最后一个名额，只有一个成功。结果见提交 bdfb261。

AI-04 统计、导出、日志与通知补发（标签 lab3-rev-3）

使用场景：要求实现名单筛选、容量与签到率统计、CSV 导出、平台汇总、日志查询与失败通知补发。

生成结果：生成 reports.py，按场次与票种汇总容量、报名、候补、剩余、签到、缺席与签到率，并提供 CSV 导出、平台汇总与日志筛选。

优点：查询覆盖主要统计口径；CSV 字段完整；平台汇总把活动、报名、用户、签到四类数据放在一起。

缺点：初稿签到率用“签到 ÷ 全部报名”，与实验一确定的“签到 ÷（签到 + 缺席）”不一致；CSV 直接写目标文件，失败会留下半截文件；导出失败没有日志；参与者也能调用导出；补发通知没

有写日志。

修改优化后的结果：修正签到率分母与零分母显示；CSV 改为临时文件加原子替换，失败清理并记日志；补上数据范围检查与补发日志。结果见提交 818a34c。

AI-05 单元测试与代码风格（标签 lab3-final）

使用场景：要求为核心规则生成单元测试，并运行 Pylint，指出代码异味与风格问题。

生成结果：生成 12 个单元测试，覆盖容量与候补递补、请求幂等、截止与重复报名、重复取消、签到唯一、场次授权、取消活动保留签到历史、结束场次标记缺席、角色越权、导出与日志、通知补发、最后名额并发；Pylint 初次检查给出若干“参数过多”“分支过多”提示。

优点：测试把需求中的异常与边界变成了可执行断言；测试为每个用例新建临时数据库，互不影响；发现并发问题时有明确的复现方式。

缺点：初稿测试偏重顺利路径，并发测试只是顺序调用两个用户，无法证明不会超卖；部分参数过多提示来自把数据库列直接作为函数参数。

修改优化后的结果：人工补上用 threading.Barrier 让两个线程同时起跑的真实并发测试，并断言只有一个成功、占用数为 1；对 Pylint 提示逐条评估，菜单分发的分支数属于界面层可以接受，参数对应数据库列，拆成对象会引入不必要的结构，保留并在 5.4 节说明。最终 12 个测试全部通过，Pylint 评分 9.92/10。结果见提交 737ee64。

4.3 大模型生成不符合预期时的处理

本次实验遇到的不符合预期主要集中在四类，处理方式可以复用。第一类是“看起来完整但缺边界”，例如初稿的报名没有截止与幂等、签到没有授权。处理方式是先把实验一的需求编号与实验二的时序步骤逐条对照，列出缺失项，再把缺失项作为新的提示重新生成，而不是直接手改一段模糊代码。第二类是“实现与设计图冲突”，例如初稿把候补塞进 registrations 表、把活动创建即成已发布。处理方式是以需求与状态图为准，先改数据模型和状态转换，再改代码，避免只改表面分支。第三类是“技术方案过度”，例如模型建议引入 ORM、连接池或消息队列。处理方式是评估实验规模后拒绝，保持标准库与单文件数据库，把复杂度留给真正需要的约束。第四类是“测试看似通过但证明力不足”，例如并发测试串行执行。处理方式是补上能证伪的断言（两个线程同时起跑、只允许一个成功），用失败用例检验测试本身。

此外还有两点具体做法：一是让模型先输出“存根 + 规约”，即函数签名、前置条件、后置条件和异常，再填充实现，这样人工可以在写代码前就纠正理解偏差；二是每次只让模型改一个用例，改完立即运行单元测试与 Pylint，避免一次生成几百行后难以定位问题。

4.4 工作流反思：能否只做检查校对

本次实验的流程可以概括为“人定约束—机出初稿—自动验证—人工审查—版本留痕”。人先确定实体、状态、接口和不变量；模型据此生成表结构、函数骨架与测试；自动验证包括运行演示、跑单元测试、跑 Pylint；人工重点审查状态转换、事务边界、权限与失败路径；每一阶段单独提交并打标签，便于回看人工改动。与实验二相比，代码阶段的自动验证更强：语法、导入、断言与静态检查都能立即给出反馈，模型犯的错更容易暴露。

但“只做检查校对”仍然不现实。模型可以生成大量可运行代码，但它不知道哪些需求尚未确认，也无法替人判断“占用数是否冗余”“候补是否应拆表”“并发用锁还是版本号”这类取舍；它还会把“看起来合理”的实现写得像已经满足需求。真正需要人做的是定义验收标准、判断边界是否覆盖、在多种实现间取舍。比较现实的分工是：模型承担初稿、样板代码、测试草稿和重复修改，人保留需求解释、模型设计、并发与安全审查以及最终验收。当需求足够形式化、测试足够强时，检查校对的比例可以提高，但责任仍由人承担。

五、代码的编译与运行结果

5.1 构建方式

Python 不需要单独的编译命令，解释器在导入模块时完成字节码编译。为保证提交的代码没有语法错误，构建时先执行 python3 -m compileall -q code，无输出表示全部模块编译通过；随后运行单元测试与 Pylint。程序没有第三方依赖，评审者只需 Python 3.10 及以上版本即可复现。

5.2 端到端运行

运行命令为 cd code && python3 main.py，首次运行会在仓库的 data 目录创建 SQLite 数据库并写入演示数据。为便于一次性展示多个角色，代码目录提供 demo.sh，使用临时数据库依次执行“参与者报名—第二位参与者报名—检票—组织者统计—管理员汇总”，不会污染正式数据库。实际运行的关键输出如下。

```
=== 1. participant1 registers ===
欢迎，王小满（参与者）
报名成功 凭证码： BWRJHP

=== 2. participant2 registers ===
欢迎，陈青（参与者）
报名成功 凭证码： 72KC7U

=== 3. checker verifies BWRJHP ===
欢迎，检票员小李（检票人员）
签到成功

=== 4. organizer statistics ===
欢迎，城南文化（组织者）
周六夜场 / 普通票：容量 2  已报名 2  候补 0  剩余 0  签到 1  缺席 0  签到率 100%

=== 5. administrator summary ===
欢迎，平台管理员（平台管理员）
活动状态： 已发布 1, 草稿 1
报名状态： 已报名 1, 已签到 1
用户角色： 参与者 2, 平台管理员 1, 检票人员 1, 组织者 1
成功签到次数： 1
```

代码 5-1 端到端演示输出（节选）

演示表明：场次容量为 2，两名参与者报名后剩余为 0；检票人员用凭证码签到后，统计中的已报名为 2（含已签到）、签到 1、签到率 100%；管理员看到的活动、报名、角色与签到汇总与数据库一致。演示开始时还会执行候补失效与场次结束检查，因此重启程序后不会留下过期的候补或未标记的缺席。

5.3 单元测试

测试命令为 python3 -m unittest discover -s tests -v。12 个测试覆盖五组规则，结果全部通过，耗时约 3.9 秒，见表 5-1。

测试类	测试	验证点
RegistrationTests	capacity_and_waitlist_promotion	满额转候补；取消后按队列递补；占用数不变
RegistrationTests	idempotent_request_returns_first_result	同一请求号只生成一条报名，返回首次结果
RegistrationTests	deadline_and_duplicate_registration	截止后拒绝；同一场次重复报名被拒
RegistrationTests	cancel_twice_is_rejected	已取消的记录不能再次取消
CheckInTests	check_in_is_unique	重复核验只提示已签到，状态不变，留两条记录
CheckInTests	checker_without_session_permission_is_rejected	未授权场次拒绝核验
EventLifecycleTests	cancel_event_keeps_checked_in_history	已签到保留，未签到改为活动取消
EventLifecycleTests	finished_session_marks_absent	场次结束标记缺席，活动结束
EventLifecycleTests	role_permissions	越权创建、审核、查看统计被拒
ReportingTests	export_and_audit_log	CSV 字段完整，导出写日志
ReportingTests	retry_failed_notification	失败通知可补发，补发后无失败项
ConcurrencyTests	test_last_seat_is_not_oversold	两个线程抢最后一个名额，仅一个成功

表 5-1 单元测试与覆盖点

5.4 代码风格检查

使用 Pylint 检查全部产品代码，命令与结果如下。

```
python3 -m pylint main.py db.py models.py auth.py events.py \
    registration.py checkin.py notifications.py reports.py

************* Module main
main.py:302:0: R0912: Too many branches (13/12) (too-many-branches)
************* Module db
db.py:146:0: R0913: Too many arguments (6/5) (too-many-arguments)
db.py:146:0: R0917: Too many positional arguments (6/5) (too-many-positional-arguments)
************* Module events
events.py:118:0: R0913: Too many arguments (10/5) (too-many-arguments)
events.py:118:0: R0917: Too many positional arguments (10/5) (too-many-positional-arguments)
events.py:165:0: R0913: Too many arguments (7/5) (too-many-arguments)
events.py:165:0: R0917: Too many positional arguments (7/5) (too-many-positional-arguments)
-----------------------------------
Your code has been rated at 9.92/10
```

代码 5-2 Pylint 检查结果

剩余 7 条都是设计类提示，而不是错误：1 条分支过多（R0912），6 条参数过多（R0913/R0917，来自三个函数）。main.py 的菜单分发函数有 13 个分支，属于界面层的命令映射；events.create_event 的 10 个参数与活动、场次、票种三张表的字段一一对应，拆成三个对象或引入构造器会掩盖“一次事务写入三张表”这一事实；db.log_action 的 6 个参数对应审计日志的六个字段。

人工评估后保留这些写法，并在报告中如实记录，而不通过禁用注释来“美化”分数。其余风格问题已在最终提交中修正，包括未使用的导入、过长行、缺少文档字符串、异常捕获后的说明等。

六、Git 远程代码管理

6.1 远程仓库

实验一、实验二、实验三使用同一个 GitHub 仓库，分别放在 lab1、lab2、lab3 分支。远程地址与当前分支状态如下。

```
origin  git@github.com:kkksc114/Software_Engineering_Lab.git (fetch)
origin  git@github.com:kkksc114/Software_Engineering_Lab.git (push)
* lab3  737ee64 [origin/lab3] code: finalize cli implementation with tests, lint fixes and demo
```

代码 6-1 远程仓库与分支跟踪

lab3 分支已经推送到 origin，本地分支与远程分支指向同一提交。实验中通过 git push -u origin lab3 建立跟踪关系，之后每次阶段性提交都推送，远程仓库可以看到完整的开发过程。

6.2 分支、提交与标签

实验三从实验二的定稿提交分出 lab3 分支，共 5 个功能提交，每个提交对应一次 AI 初稿或一轮人工修改，并打注释标签，见表 6-1。

标签	提交	内容
lab3-ai-draft	845bf0a	AI 生成的内存原型初稿（216 行单文件）
lab3-rev-1	9c5f238	人工重构：SQLite 持久化、活动生命周期、角色与演示数据
lab3-rev-2	bdfb261	人工重构：报名、候补递补、检票与通知事务
lab3-rev-3	818a34c	人工重构：统计、CSV 导出、审计查询与通知补发
lab3-final	737ee64	单元测试、Pylint 修正、演示脚本与文档

表 6-1 实验三标签与提交

提交信息统一使用“类型: 内容”的格式，类型包括 code、test、docs、chore。AI 初稿与人工修改不混在同一次提交中：lab3-ai-draft 只包含模型生成的 216 行原型，后续每个 rev 标签只包含本轮人工修改的内容，便于逐段审查。

6.3 人工修改的追溯

评审者可以用两条命令查看全部人工改动：git log --oneline --decorate lab3 查看阶段顺序，git diff lab3-ai-draft..lab3-final -- code 查看从 AI 初稿到最终提交的代码差异。每次 rev 提交的消息都写明了本轮实现了什么；标签使用注释标签，标签说明记录阶段目标。这样做的好处是，报告中的每个“人工修改”结论都能对应到具体 diff，而不是只在文字里声明。

七、总结与不足

本次实验把实验二的领域类图、服务类图、状态图与时序图落实为约 1967 行 Python 代码，覆盖四类角色、活动六态生命周期、报名与候补、检票、统计导出、通知与审计。实现用 SQLite 事务、部分唯一索引与显式事务块保证“不超卖、不重复报名、重复请求返回首次结果”，用 12 个单元测试把这些规则固定下来，Pylint 评分 9.92/10，全部提交已推送到 GitHub 的 lab3 分支并打上 5 个阶段标签。

实现与实验二设计存在几处有意差异：服务类改为模块函数；占用数改为实时计数；乐观锁版本号改为 SQLite 写锁；报名七态中的候补状态拆到 waitlist 表；界面改为命令行。这些差异都在第二章说明了理由，实验三的补充说明也允许设计与实现不一致。

不足主要有五点。第一，界面是命令行，实验二的 HTML 界面没有接入真实服务；第二，SQLite 是单机数据库，并发测试只验证了同机双线程，没有覆盖多进程或更大压力；第三，通知只记录状态，没有接入真实邮件或短信渠道，因为实验一尚未确认渠道；第四，未做性能压测，实验一提出的查询时延指标不能在本次实验中宣称已满足；第五，Pylint 仍有 7 条设计类提示，属于权衡后保留。后续实验可以在此基础上做性能分析、故障注入与测试覆盖率统计。

最大的收获是：UML 中的状态、关联和时序只有落到约束与事务上才有意义；大模型能迅速铺开代码骨架和测试草稿，但它不会主动发现需求冲突，也不对边界负责。把每次生成都当作待审查的初稿，用小步提交、可执行测试和静态检查去证伪，是本次实验中比“让模型写更多代码”更重要的经验。

参考文献

[1] Python Software Foundation. Python 3.10 Documentation [EB/OL]. https://docs.python.org/3.10/

[2] SQLite Consortium. SQLite Documentation [EB/OL]. https://www.sqlite.org/docs.html

[3] Pylint contributors. Pylint Documentation [EB/OL]. https://pylint.readthedocs.io/

[4] PlantUML. PlantUML Language Reference Guide [EB/OL]. https://plantuml.com/zh/

[5] Moriarty K, Kaliski B, Rusch A. PKCS #5: Password-Based Cryptography Specification Version 2.1: RFC 8018 [S]. IETF, 2017.

[6] ReportLab. ReportLab User Guide [EB/OL]. https://www.reportlab.com/docs/reportlab-userguide.pdf

---

附录 A 提交文件结构

```
实验3_241880173_陆知渔/
├── 实验报告.pdf
├── UML/
│   ├── UML类图.puml / UML类图.png
│   ├── UML服务类图.puml / UML服务类图.png
│   ├── UML组件图.puml / UML组件图.png
│   ├── UML用例图.puml / UML用例图.png
│   ├── UML活动图.puml / UML活动图.png
│   ├── UML状态图.puml / UML状态图.png
│   └── UML时序图.puml / UML时序图.png
└── code/
    ├── main.py
    ├── db.py
    ├── models.py
    ├── auth.py
    ├── events.py
    ├── registration.py
    ├── checkin.py
    ├── reports.py
    ├── notifications.py
    ├── demo.sh
    ├── README.md
    └── tests/test_services.py
```

附录 A-1 提交目录

附录 B 运行与检查命令

```
# 编译检查（字节码编译全部模块）
python3 -m compileall -q code

# 启动程序
cd code && python3 main.py

# 端到端演示（临时数据库）
cd code && bash demo.sh

# 单元测试
cd code && python3 -m unittest discover -s tests -v

# 代码风格检查
cd code && python3 -m pylint main.py db.py models.py auth.py events.py \
    registration.py checkin.py notifications.py reports.py

# 查看人工修改
git diff lab3-ai-draft..lab3-final -- code
```

附录 B-1 常用命令